

How the **Love Button** was built

Tap a widget on your home screen, her phone buzzes with a distinct sound.
That is the entire product. Everything below is what it costs to make that sentence true at 3am on a Xiaomi phone that has been asleep for six hours.

~7,000 lines

Kotlin + TypeScript

75 server tests

17 Android test files

£0 / month

4 messages, forever

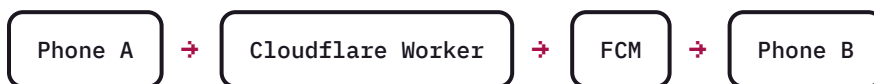
00 - ORIENTATION

What it is, and the one constraint that shapes everything

Four fixed messages. Four home-screen widgets, one per message, each with its own pixel icon and its own notification sound. The sender's tile lights up when the message is delivered, and again when she opens it. No message list, no chat, no counter.

The constraint that decides every other design choice: **two phones cannot talk to each other**. Neither has a fixed address, both spend most of their lives asleep under Android's Doze, and waking a sleeping Android phone from outside requires Firebase Cloud Messaging. FCM will only accept a push signed by a Google service-account private key — and that key can push to *any* device in the project, so it can never ship inside an APK.

Hence a server. It exists for exactly one reason: to be the single place that key lives. Everything else it does (deciding who may send what to whom, storing receipts) is a consequence of already having to exist.



Two rules worth memorising before reading code

- **The push carries a number, never words.** `msg_id: 3`. The receiving app maps `3` → text, icon and sound locally. The actual messages therefore never transit Google's servers, and adding a fifth message is an app-only change.
- **There is no "recipient" field anywhere.** With exactly two people, the recipient is arithmetic: `3 - from_person`. There is no lookup to get wrong and no request field to tamper with.

STACK

Client: Kotlin · Compose (screens) · Glance (widgets) · WorkManager · DataStore · OkHttp · Firebase

Messaging.

Server: TypeScript · Hono · Cloudflare Workers · D1 (SQLite) · KV.

Free tiers only. No card on file.

01 – THE ROUND TRIP

One tap, traced end to end

This is the spine of the whole system. Read it once now; every file in sections 02 and 03 is a step on this list.

- 01 **PHONE A** **Tap.** `SendAction` paints the tile `SENDING` immediately, then calls `beginSend()`. Nothing blocking — an `ActionCallback` runs on the launcher's clock.
- 02 **PHONE A** **The tap mints its own identity.** `beginSend` generates a `send_id` UUID and a timestamp, writes them to `CurrentSend`, then enqueues two `WorkManager` jobs: `SendWorker` (network-constrained) and `TimeoutWorker` (deliberately unconstrained, delayed 20s).
- 03 **PHONE A** **`SendWorker` records the mapping before the request leaves.** `send_id` → `appWidgetId` goes into `PendingSends` first, because a receipt can beat the response back.
- 04 **PHONE A** `POST /v1/send` with `{msg_id, send_id}` and `Authorization: Bearer <token>`. That is the whole body.
- 05 **WORKER** **`requireDevice` resolves identity.** SHA-256 the presented token, look up `devices.auth_hash`. From here on, `c.get("device")` is the only source of truth for who is calling.
- 06 **WORKER** Rate ceiling checked (500/hr per person), `msg_id` checked against `{1,2,3,4}`, `send_id` checked as a real UUID — it becomes a primary key, so it is validated, not trusted.
- 07 **WORKER** **INSERT OR IGNORE into `sends`.** Row written *before* the push, because it is what a receipt correlates against later. `changes === 0` ⇨ 409 duplicate. That check is the only one that cannot race.
- 08 **WORKER** **Get a Google access token.** Cached in KV for 55 min. Cold cache ⇨ build an RS256 JWT from the service-account PEM with `WebCrypto`, exchange it at `oauth2.googleapis.com`. The cache is not an optimisation — RSA signing sits near the Workers free tier's 10ms CPU ceiling.
- 09 **WORKER** **Fan out.** `SELECT ... WHERE person = 3-from AND fcm_token IS NOT NULL`, one HIGH -

priority data-only push per token. HIGH is the exemption that punches through Doze.

- 10 **WORKER** Returns `{send_id, delivered, sent_at}`. Always 200, even when `delivered` is 0 — “she has no active device” is a different and more useful message than a failure. `sent_at` is the clock both phones will order by.
- 11 **PHONE B** **`PushService.onMessageReceived`**. Because the Worker never sends a `notification` block, the message lands in app code instead of the system tray — which is what lets the app pick the channel, and therefore the sound.
- 12 **PHONE B** Post the notification on channel `msg_3` → its own sound fires. *Then* report `delivered`, never before: a receipt for a notification that failed to post would be a lie.
- 13 **PHONE B** **Is anyone actually looking?** `couldBeLookingNow()` = screen interactive *and* keyguard unlocked. Yes → report `seen` now. No → park the id in `UnseenSends`; `UnlockReceiver` drains it at the next unlock.
- 14 **WORKER** **`POST /v1/receipts`**. Only `row.to_person === device.person` may acknowledge. Writes `use COALESCE(delivered_at, ?)` so state is monotonic by construction and a replay cannot move it backwards. Then a `NORMAL` -priority push back to the sender.
- 15 **PHONE A** **The tile advances.** `PushService` updates `currentSend` first (the app screen keeps history), then looks up the widget — only if the 20-second window is still open. Outline → half → filled → blush → gold.

WHY STEP 02 MINTS THE ID ON THE PHONE

If the server minted `send_id`, the `send_id` → `widget` mapping could only be written *after* the response came back — and a receipt from a fast phone can arrive before that response. Minting it client-side makes the race **unreachable** rather than merely handled. That is the single highest-leverage design decision in the app.

02 — THE TWO PLATFORMS

What Firebase and Cloudflare actually do here

Neither is used as a “backend as a service”. Firebase is used for exactly one product, and Cloudflare is used as a place to run about 900 lines of TypeScript with a database attached. Understanding what each one *is* makes the setup steps in section 09 obvious rather than ritual.

Firebase — really just FCM

firebase — really just fcm

A Firebase project is a container. This app touches one product inside it: **Cloud Messaging**. No Firebase Auth, no Firestore, no Analytics, no Crashlytics. Firebase is here because it is the only way to wake a sleeping Android phone from the outside — Google runs one persistent socket per device from Play Services, and FCM is the front door to it.

Three separate identities come out of that one project, and confusing them is the most common way to get stuck:

IDENTITY	WHAT IT IS	WHERE IT LIVES
google-services.json	The client identity. Package name, project id, and an API key restricted to that package. The Gradle plugin bakes it into the APK at build time.	Ships inside the APK, so it is not really secret. Kept out of the repo by choice, not necessity.
Service account key	The server identity. A JSON file containing an RSA private key that can push to <i>any</i> device in the project.	One place only: a Cloudflare Worker secret. Never in git, never in the APK. This is the one that matters.
Registration token	The device address . Minted by Play Services on each install, ~160 characters, and it rotates .	<code>devices.fcm_token</code> in D1. If the server holds a stale one, pushes go nowhere and nothing reports an error — which is why <code>/v1/devices</code> is called on every launch.

WHAT A PUSH PHYSICALLY DOES

1. The Worker signs a JWT with the service-account private key and trades it at `oauth2.googleapis.com` for a one-hour access token.
2. It `POST`s to `fcm.googleapis.com/v1/projects/<project_id>/messages:send` with `{ message: { token, data, android: { priority } } }`.
3. Google routes it down Play Services' existing socket to that device.
4. Play Services hands it to the app's `FirebaseMessagingService`.

Two flags decide everything about how that feels. **priority: HIGH** is the exemption that punches through Doze and wakes a sleeping phone — messages are sent HIGH; receipts back to the sender are NORMAL, because a receipt is not urgent and NORMAL costs far less battery and quota. And the payload is **data-only**: the moment you include a `notification` block, Android renders it itself in the system tray, which bypasses the app's per-message channels and therefore its per-message sounds. That single omission is what makes the product work.

THE LIMIT OF STORE-AND-FORWARD

FCM queues messages for a device that is *unreachable*. It does **not** queue for a reachable phone whose app has been force-stopped: delivery is accepted at the Play Services layer, Play Services finds no app to hand it to, and drops it. No TTL setting changes this, and reopening the app replays nothing. Verified on hardware. There is deliberately no catch-up endpoint — four stale buzzes arriving an hour late is worse than none, because the product is the moment rather than the message.

Cost: FCM is free and unmetered on the Spark plan. No card. The real constraints are that FCM throttles per device at roughly a few hundred messages a minute, and Android itself drops notifications from an app posting too rapidly — both floors sit *below* our own 500/hour ceiling, so rapid-fire sends get dropped by the platform before they get dropped by us.

Cloudflare — a function at the edge, with a database bolted on

A Worker is not a server. There is no host, no port, no process to keep alive, no connection string, and effectively no cold start. You upload a JavaScript module; Cloudflare runs it in a V8 isolate in whichever datacentre is nearest the caller. Four pieces are in play:

PIECE	WHAT IT IS	USED FOR
Workers	The runtime. Browser-standard APIs (<code>fetch</code> , <code>crypto</code> , <code>TextEncoder</code>) — no Node built-ins, which is why <code>crypto.ts</code> wraps <code>WebCrypto</code> .	All five endpoints.
D1	SQLite at the edge, addressed through a binding. No cross-statement transactions in the usual sense — <code>batch()</code> is how two writes land together.	<code>devices</code> and <code>sends</code> .
KV	Key-value with TTLs, eventually consistent .	The cached Google access token, and the per-IP enrol counter — where a slightly stale count is acceptable and is stated as such in the code.
Cron Triggers	A scheduler that calls the module's <code>scheduled()</code> export.	The nightly retention sweep at 03:17 UTC.

BINDINGS — THE IDEA WORTH TAKING AWAY

D1 and KV are not reached by URL and password. They arrive as *objects* on `env` , wired up by `wrangler.toml` :

```
[[d1_databases]]
binding = "DB"           # becomes env.DB in code
database_name = "love-button"
database_id = "2bf232d6-..."

[[kv_namespaces]]
binding = "TOKEN_CACHE"  # becomes env.TOKEN_CACHE
id = "939528e7..."

[triggers]
crons = ["17 3 * * *"]  # 03:17 UTC, not on the hour — see below
```

Because the platform injects them, **there is no database credential anywhere in this project**. That is not a

convenience; it removes an entire category of secret from the threat model.

THE FREE-TIER NUMBERS THAT SHAPED THE CODE

- **100,000 requests/day.** Two people tapping a widget will not notice this exists.
- **10ms CPU per request.** This one is real. An RSA signature sits close to that ceiling, which is *the* reason the Google access token is cached in KV rather than minted per send. Receipts double the request volume, so it matters more here than it would in a send-only design.
- **Secrets are write-only.** `wrangler secret list` returns names only. There is no command, flag, or dashboard page that reads a value back — by design, not an oversight. A backup script therefore cannot export them; they must be captured from their original source, once.
- **Cron at 03:17, not 03:00.** Schedules on the hour cluster with everyone else’s on the platform and get dispatched less promptly. The exact minute is irrelevant to correctness — the purge is defined by a cutoff, not a cursor, so a late or skipped run just deletes slightly more the next night.

A WRINKLE WORTH KNOWING

The `workers.dev` subdomain is derived from the account’s email local-part and Cloudflare exposes no way to change it — the API replies “Account already has an associated subdomain”. Since this repo is public, the deployed URL is kept out of git entirely and read from `local.properties` at build time instead.

WHAT LIVES WHERE, IN ONE TABLE

THING	HOME	IN GIT?
Service-account private key	Worker secret <code>FCM_SERVICE_ACCOUNT</code>	Never
Two enrolment codes	Worker secrets + your password manager	Never
Release keystore	Local file + encrypted backup	Never
Deployed Worker URL	<code>local.properties</code>	No (public repo)
<code>google-services.json</code>	Local file, baked into the APK	No, by choice
D1 id, KV id, Firebase project id	<code>wrangler.toml</code>	Yes — identifiers, not credentials
Both people’s names	<code>wrangler.toml</code> vars	Yes

03 – THE SERVER

Fourteen small files on Cloudflare Workers

A Worker is a function at the edge with no server to run, no ports and no connection

strings. D1 (SQLite) and KV arrive as injected *bindings* — objects on `env`. Secrets arrive the same way, set once with `wrangler secret put` and unreadable afterwards.

CORE

FILE	WHAT IT DOES, AND THE REASON IT EXISTS
<code>index.ts</code>	Mounts four Hono routers under <code>/v1</code> , plus an unauthenticated <code>/health</code> . Exports an object rather than the bare app because a Worker can only expose a <code>scheduled</code> (cron) handler that way. Custom 404/500 handlers so every response is JSON — Hono's defaults are <code>text/plain</code> .
<code>env.ts</code>	Types only: the two bindings, three plain vars, three secrets, and the <code>DeviceRow</code> the auth middleware attaches to the request.
<code>http.ts</code>	<code>fail()</code> — one error shape for the entire API, so the Android client parses one thing. <code>nowSeconds()</code> , and a <code>readJson</code> that returns <code>null</code> instead of throwing.
<code>crypto.ts</code>	WebCrypto wrappers (Workers have no Node <code>crypto</code>). <code>randomToken()</code> = 32 secure random bytes as hex. <code>secureEquals()</code> hashes both sides first, then XORs 32 bytes in a fixed-length loop — leaking neither content nor length through timing.
<code>auth.ts</code>	The <code>requireDevice</code> middleware. Hash the presented bearer token, find the matching row. Note what is absent : nothing reads an identity from a request body.
<code>limits.ts</code>	500 sends/hour per person (counted in D1) and 5 enrolment attempts/hour per IP (a KV counter with a 1-hour TTL). A circuit breaker against a compromised device, not a usage limit.
<code>messages.ts</code>	Six lines: the allowlist <code>{1,2,3,4}</code> . The server knows the numbers and nothing else.
<code>retention.ts</code>	<code>DELETE FROM sends WHERE sent_at < cutoff</code> , 7 days, run nightly by cron at 03:17 UTC. Without it, a database that promises not to be a history of every message becomes exactly that.

TALKING TO GOOGLE

FILE	WHAT IT DOES
<code>google-oauth.ts</code>	Machine-to-machine auth with no human present. Parse the service-account JSON, convert its PEM private key to bytes, build a JWT claiming <code>iss = client_email</code> and <code>scope = firebase.messaging</code> , sign it RS256 with WebCrypto, POST it to Google, get back a 1-hour access token. Cached in KV at 55 minutes.
<code>fcm.ts</code>	One <code>POST</code> per device token to the FCM v1 endpoint. Never includes a <code>notification</code> block — doing so hands rendering to the system tray and bypasses the app's per-message channels, and therefore its per-message sounds, which is the whole product. Classifies the response into <code>ok</code> / <code>unregistered</code> / <code>error</code> .

The Worker used to delete a device row when FCM returned `UNREGISTERED` or `INVALID_ARGUMENT`. But FCM returns `INVALID_ARGUMENT` for a malformed **request** as readily as for a bad token — and a malformed request fails identically for *all* of her tokens, so every one of her device rows got deleted at once. Her bearer token's row went with them, so her app 401'd on everything and could only recover by re-enrolling by hand.

The fix is one line and one principle: trust only the nested `details[].errorCode === "UNREGISTERED"`, never the outer `error.status`, which is derived from the HTTP code and can equally mean a mistyped project id. **A token that keeps failing is a far cheaper failure than one that locks someone out.**

THE FOUR ROUTES

ENDPOINT	AUTH	BEHAVIOUR
POST <code>/v1/enroll</code>	enrol code	Compares the code against both secrets with no early return, so timing reveals nothing. Mints a device id + a bearer token, stores only the token's hash, and returns the raw token once — never recoverable again. Deletes any prior row with the same <code>fcmtoken</code> (same physical phone re-enrolling). A body with no <code>fcmtoken</code> creates a <i>send-only</i> device: it can send, but <code>/v1/send</code> 's <code>WHERE fcm_token IS NOT NULL</code> means it never receives.
POST <code>/v1/devices</code>	bearer	Refreshes a rotated FCM token. Called on every launch. If the server holds a stale token, pushes go nowhere silently.
DELETE <code>/v1/devices</code>	bearer	Sign out. Deletes this device only.
POST <code>/v1/send</code>	bearer	Steps 06–10 of the trace above.
POST <code>/v1/receipts</code>	bearer	Step 14. Idempotent and monotonic; <code>seen</code> also backfills <code>delivered_at</code> when she opened the notification before the delivered receipt landed. 200 regardless of whether the reverse push succeeded — the client cannot act on that.

THE WHOLE SCHEMA

```
-- No users table (names are wrangler vars). No invites table
-- (there is no pairing flow). Two tables is the entire database.

CREATE TABLE devices (
  id          TEXT PRIMARY KEY,
  person      INTEGER NOT NULL CHECK (person IN (1,2)),
  auth_hash   TEXT NOT NULL UNIQUE,    -- SHA-256 of the bearer token
  fcm_token   TEXT,                   -- NULL = send-only device
  label       TEXT,
  created_at  INTEGER NOT NULL, updated_at INTEGER NOT NULL
);

CREATE TABLE sends (
```

```

id            TEXT PRIMARY KEY,      -- minted by the phone
from_person   INTEGER NOT NULL,
to_person     INTEGER NOT NULL,
msg_id        INTEGER NOT NULL,
sent_at       INTEGER NOT NULL,      -- the shared ordering clock
delivered_at  INTEGER, seen_at INTEGER
);

```

"There is no way to add a third person" is not a convention here. It is `CHECK (person IN (1,2))`, enforced by SQLite.

04 – THE APP

Six packages, and why each one exists

Two UI toolkits, because a widget is not a screen. Compose draws the app; Glance draws the four tiles, and a Glance widget is `RemoteViews` under the hood — another process renders it, which is why nothing arbitrary works there.

DATA/ – STATE AND THE WIRE

FILE	ROLE
Messages.kt	The catalogue: <code>id</code> → text, notification channel, sound resource. Lives in the app, never on the server.
Prefs.kt	<code>DataStore</code> . Three values written once at enrolment: bearer token, person number, partner name. Nothing else. On a non-rooted device, app-private storage <i>is</i> the boundary that matters.
ApiModels.kt	kotlinx.serialization request/response types. <code>SendRequest</code> has no recipient field, deliberately. <code>sentAt</code> is defaulted so an app talking to an older Worker still parses.
LoveButtonApi.kt	OkHttp + serialization, no Retrofit. Three endpoints; being able to read the whole client in one sitting beats the boilerplate a framework saves. Never logs the token — Logcat is readable by anyone with adb.
CurrentSend.kt	The app screen's single record. Exactly one message, whoever sent it. Persisted, not in-memory, because MIUI kills the process routinely and the screen must show the last outcome on a cold open. This is where the shared-bubble ordering lives.
PendingSends.kt	<code>send_id</code> → <code>appWidgetId:timestamp</code> , so a receipt can find its tile. Also defines <code>PENDING_WINDOW_MS = 20_000</code> and the single <code>windowClosed()</code> predicate that three separate places must agree on.
UnseenSends.kt	Messages that arrived while the phone was asleep, waiting for the next unlock. <code>drain()</code> reads and clears in one edit, so two overlapping unlock broadcasts cannot report the same id twice.

WORK/ – EVERY NETWORK CALL, WITHOUT EXCEPTION

Nothing calls the network inline. WorkManager persists jobs across process death and retries on reconnect; on MIUI that is the difference between a tap that eventually lands and one that vanishes.

FILE	ROLE
Sending.kt	<code>beginSend()</code> — the one definition of what a tap means. Both the widget and the app's own buttons go through it. Mints the id and the timestamp <i>at the tap</i> , so a retry keeps the same id and the 20-second clock is measured from when you actually pressed it.
SendWorker.kt	Network-constrained. Performs the send, advances the tile, holds <code>SENT</code> for the full 20s window because a receipt may still arrive.
TimeoutWorker.kt	Delayed 20s, no network constraint — the whole point. The send it shadows is held back until there <i>is</i> a connection, so a timeout waiting on the same thing would never fire in exactly the case it exists for: a tap made with no signal.
ReceiptWorker.kt	Reports <code>delivered</code> / <code>seen</code> . A rejected receipt (403, 404) returns success — retrying cannot improve it. Only a thrown network error retries.
RegisterTokenWorker.kt	Pushes a rotated FCM token to the server.

PUSH/ — RECEIVING

FILE	ROLE
PushService.kt	Two branches: <code>type: "msg"</code> posts a notification and reports receipts; <code>type: "receipt"</code> advances the tile and never notifies — a phone that buzzes when she reads a message is a phone nobody wants.
Notifications.kt	Creates the four channels at startup. Builds each channel's sound URI from the resource name , never its numeric id — ids renumber when you add resources, a channel's sound is frozen at creation, and Android's response to an unresolvable sound is silence with no error anywhere.
Presence.kt	Four lines that decide whether "seen" is true: screen interactive <i>and</i> keyguard unlocked. Neither alone is sufficient — an interactive screen behind the keyguard means the phone woke for the notification, not that anyone read it.
UnlockReceiver.kt	Turns "she picked her phone up" into a <code>seen</code> receipt. Registered in code, never in the manifest : <code>ACTION_USER_PRESENT</code> is not on Android's implicit-broadcast exemption list, so a manifest-declared receiver would look correct and never run.

WIDGET/ — FOUR TILES, ONE IMPLEMENTATION

Four separate *registrations* are required so the launcher lists four entries with their own preview and label. Four *implementations* are not: `MessageWidget(msgId)` is abstract, and `LoveWidget` / `ThinkingWidget` / `MissWidget` / `CallWidget` carry only an id.

- `WidgetState.kt` — the six states, their hold durations, and `ladderRank`, written out explicitly rather than using the enum's ordinal so a reorder for readability cannot silently change which receipt wins a race.

- `WidgetStateWriter.kt` — the only place a tile is written. Addressed by platform `appWidgetId`, not `GlanceId`, because this crosses a process boundary and the worker may run after the host that created it was killed.
- `WidgetArt.kt` — the shared palette and the state → drawable map, used by the widget *and* the in-app guide, so the guide is structurally incapable of showing something the widget does not.
- `BootReceiver.kt` — after a reboot the host recreates tiles but nothing re-renders them, so they sit on the loading layout and a tap falls through to the launcher's default. This redraws all four.

UI/ — THE STICKER BOOK

One theme, not two skins: every element is a sticker — hard ink keyline, flat colour, shallow offset shadow. Four screens (`Enrol`, `Home`, `Setup`, `Guide`), and the navigation model is literally "enrolled or not".

`StateCopy.kt` holds one set of words for all six states, split into words and a kaomoji face so the home screen can animate the face while the guide renders the whole line. `PixelArt.kt` and `PixelLadder.kt` redraw the widget icons on a Compose `Canvas`, per pixel, so the in-app hero can animate what a `RemoteViews` tile never could.

DEVICE/ — MIUI, TREATED AS A FIRST-CLASS FEATURE

`DeviceSetup.kt` opens OEM settings screens by component name. MIUI's component names differ between versions and are part of no public API, so every one goes through a helper that catches

`ActivityNotFoundException` and `SecurityException` and falls back to the app's own settings page. A missing OEM activity must never crash the app.

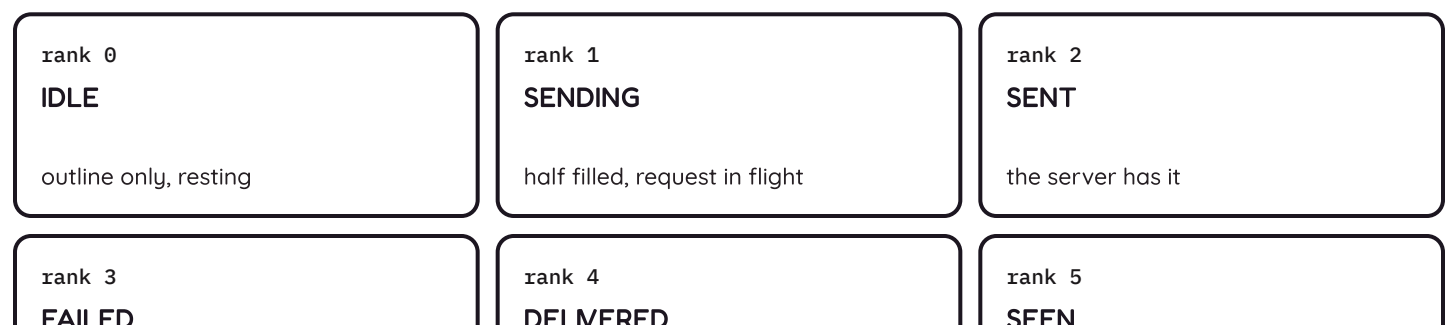
05 — FIVE HARD PROBLEMS

Where the real engineering is

If you only read one section, read this one. Everything else is plumbing you could re-derive; these are the five things that were *wrong on hardware first* and then fixed.

1 • The ladder only moves forward

The recipient reports `delivered` and `seen` as two independent `WorkManager` jobs, which run concurrently. On real hardware, `seen` was observed arriving **54ms before** the `delivered` for the same send. Written in arrival order, that late `delivered` undid the `seen` and the sender's screen stuck on "it buzzed her phone", never reaching gold.



greyed, 3s hold

it buzzed her phone

gold ring, terminal

The fix is `advancesTo()` : a write lands only if `next.ladderRank > current.ladderRank` . Note where **FAILED** sits — above **SENT** so it can follow **SENDING** (or a failed send would say "sending..." forever), and below the receipts because *a receipt can only exist if the send reached the server*, which is better evidence than this phone's own failed request.

2 · A tap with no signal

Three separate mechanisms, because a widget and a screen fail differently:

- **The record** is written at the tap, before any network work, so the screen acknowledges the touch instantly.
- **The screen** derives its timeout on every read (`displayState()`) rather than writing "failed" at the 20-second mark. A write that has to survive twenty seconds on MIUI often will not happen at all — and deriving it fixes the cold open for free: a send from an hour ago that nothing came back for is grey the moment you open the app.
- **The tile** cannot derive anything — it only repaints when something pushes an update to it — so `TimeoutWorker` exists purely to unstick it, with no network constraint.

And `SendWorker` checks `windowClosed(tappedAt)` before sending at all: a request that finally got a connection 40 minutes later is **abandoned, not retried**. The sender was already told it did not happen; buzzing her phone now would contradict that.

3 · Two clocks pretending to be one

Both phones show the same "bubble" — one shared latest message. For them to agree they must reach the same answer from the same facts, so the ordering key is the **server's** `sent_at` , compared only ever against another server clock. Comparing it to the phone's own tap time would be two clocks pretending to be one, and the bubbles would disagree exactly when it mattered.

`sent_at` is in *seconds*, so two sends moments apart routinely carry the same value. "Strictly newer" alone would leave each phone holding its own message. The tie is broken by `send_id` — arbitrary, but a total order that both phones can compute identically, which is the only property required.

OBSERVED, THEN FIXED

A send of your own has no server timestamp while it is in flight, so an arriving message wins unconditionally — *even one the server stamped earlier*. Her phone, whose own send already had a timestamp, correctly kept the newer one, and the two bubbles settled on different messages.

`markSentAt()` now reclaims the bubble from anything genuinely older when the response lands. Caught on hardware before it was written down.

4 · Two writers on one tile is a coin toss

`SendWorker` 's hold and `TimeoutWorker` 's deadline fire at the same millisecond and disagree about what should happen. So there is exactly one writer: `SendWorker` waits the window out only to forget the pending entry, and `TimeoutWorker` owns the tile. Similarly, `clearWidgetStateIf(expected)` means only the coroutine whose own write is still on screen may clear it — a `delivered` at `t=0` would otherwise wipe a gold `seen` tile at `t=4000`, a full second early, having no idea a newer receipt had already overwritten it.

5 · One irreversible decision

A notification channel's sound is frozen at creation and can never be changed. Changing it means deleting the channel, which visibly resets her notification settings. So the four real channel ids (`msg_1...msg_4`) were kept *unburnt* during development behind a throwaway `dev_buzz_v1` channel, and only created once the four sounds were final. Startup still deletes the dev channel; creating a channel that already exists is a no-op, so `ensureChannels()` is safe on every launch.

Unfixable, and therefore documented

- **Force-stopping the app kills FCM delivery** until it is reopened. This is how Android works. First thing to check when messages stop.
- **Both phones are Xiaomi**, where autostart is off by default for sideloaded apps and battery saver defaults to "Restricted". The Delivery setup screen walks through each setting and re-checks what it can on every launch.
- **China ROMs have no Google Play Services**, so FCM does not exist there at all.
- **Widget updates are IPC** — slow and rate-limited by the launcher. This is precisely why a six-state ladder replaced a multi-frame animation.

06 — THE PIXEL PIPELINE

The icons are code, not assets

Each icon is authored as an ASCII grid, because pixel art *is* a grid — editing the picture means editing the picture. A Python script turns each grid into Android `VectorDrawables`, one per state, plus a Kotlin object the app reads at runtime.



Four characters: `X` solid · `s` shine · `o` hole · `.` empty. Everything else is **derived**, never authored — and that is the point of the whole exercise.

- **The outline is derived** as "a solid cell touching an empty cell or the border". The Kotlin `Canvas` renderer applies the identical rule, which is what stops the animated in-app heart and the static widget heart from disagreeing.
- **The shade is derived from each state's own fill** ($\times 0.72$), not pinned to the border colour. The border is *lighter* than the fill while a send is in flight and *darker* once it lands, so a fixed shade would flip from shadow to highlight halfway up the ladder.
- **Holes are inside the silhouette** and cast no outline. Treating them as outside ringed every eye and every mouth in dark border colour — which is why the speech bubble's face used to read as uncanny.

The palette itself lives in exactly one place, `PixelPalette`, as ARGB ints rather than Compose `Color` so that Glance, Canvas and the in-app guide can all read it without any of them dragging in the others' dependencies.

07 — SECURITY MODEL

Three invariants and a public repo

The repository is public. That is only safe because of a threat model written before the code, and three invariants that each have a test.

1. **The sender is the authenticated device** — never read from a request body.
2. **The recipient is derived server-side** as `3 - from_person`.
3. **Only the recipient of a send may acknowledge it.** The sender holds the `send_id` by definition — he generated it — so without this check he could forge a `seen` for his own message and the widget would light up for something she never opened.

What makes a leak survivable

- The service-account key exists in exactly one place: a Cloudflare Worker secret. Worker secrets are **write-only** — `wrangler secret list` returns names only, and no dashboard page reads a value back. That is by design, and it means a backup script cannot export them; they must be captured from their original source once.
- Only SHA-256 *hashes* of device tokens are stored, so a database leak yields no working credentials.
- `/v1/send` takes a message id and nothing else — **there is no field in which to name a victim**.
- Enrolment codes are reusable by design (there is no pairing flow), which is what makes "test on a second phone, then hand over" work. The trap: rotating a code does *not* revoke tokens already issued. To cut off a device you delete its row.

`.github/hooks/pre-commit` refuses any commit whose diff adds a `-----BEGIN PRIVATE KEY-----` block or a `"type": "service_account"` line, and refuses the keystore, `keystore.properties`, `local.properties`, `.dev.vars` and `google-services.json` by name. This is not a substitute for `.gitignore` — it catches what `.gitignore` cannot: a key pasted into a file that *is* meant to be committed. Git does not enable hooks for you: `git config core.hooksPath .github/hooks`, once per clone.

Two files are deliberately absent from the repo and fail the build loudly rather than quietly: `google-services.json`, and `local.properties` with `apiBaseUrl`. Defaulting the URL to a placeholder would produce an APK that installs cleanly and then fails at enrolment with a network error — sending the reader off to debug a Worker that is fine.

And the release keystore is not in the repo and must never be. Losing it means the app can never be updated in place again: Android refuses an install whose signature changed, so the only way forward would be an uninstall and a re-enrol.

08 — TESTS & THE GATE

What is worth automating, and what is not

LAYER	HOW
Worker	<code>vitest</code> with <code>@cloudflare/vitest-pool-workers</code> — the suite runs inside the <i>real</i> workerd runtime against an in-memory D1, with migrations read from <code>migrations/</code> and applied per run. No hand-set-up database to drift. Secrets are replaced by test doubles in <code>vitest.config.ts</code> .
Android logic	JUnit + Robolectric + OkHttp MockWebServer. The pure, testable parts are pulled out on purpose: <code>receivedWins()</code> , <code>advancesTo()</code> , <code>windowClosed()</code> , <code>isBorderCell()</code> , the sway arithmetic. Anything with a subtle rule gets a function so it can have a test.
Widgets & sounds	By hand, on hardware. Automated testing of Glance and OEM notification behaviour costs far more than it returns for a two-person app. Stated as a decision, not an omission.
The overnight gate	<code>scripts/overnight-check.sh</code> . A required gate, not an optional check.

Why the overnight test is the one that matters

On MIUI, a loop that works while you are staring at the phone may not work at 3am. So the script sends to *both* phones from the laptop — neither handset is woken, unlocked, or opened, because a phone whose app was killed overnight revives the instant you touch it, which would hide the exact failure the gate exists to catch.

It used to read each phone's bearer token off the handset with `run-as`. That stopped working at 1.0, because a release APK is not debuggable — *and it should not be*: the alternative is shipping a publicly downloadable build whose private data any adb session can read. The replacement is the **send-only device**: a row with a `NULL fcm_token`, minted once with the enrolment code. It can send as a person but can never receive, so it cannot steal a push from a real phone. No new authority; the gate survives the release build.

```
# enable the hook first – git will not do it for you
git config core.hooksPath .githooks

./gradlew :app:assembleDebug
./gradlew :app:testDebugUnitTest
cd server && npm install && npx vitest run
```

09 – STANDING IT UP

Every step that needs a browser login or a phone in your hand

Five blocks. Each ends with a check that proves it worked — if a check fails, stop there rather than continuing, because every failure below this line is silent. None of this can be automated away; it is the part that needed a human, and it is the part you have not done yourself yet.

Block A · Back up first

Everything below produces values you paste into files. Back up the clean state before you start editing it.

```
gh auth login                # browser + one-time code
gh repo create LovieApp --public --source=. --remote=origin --push

# and immediately, once real files land on disk:
git config core.hooksPath .githooks
git status --short           # google-services.json must NOT be listed
```

Block B · Firebase

Free forever on the Spark plan. No card.

1. **Create the project** at `console.firebase.google.com` . Any name. Turn Google Analytics *off* — nothing here uses it.
2. **Add an Android app**. The package name must be exactly `com.lovebutton.app` . Skip the SHA-1 field; nothing here uses Google sign-in.
3. **Download `google-services.json`** → `app/google-services.json` . Already gitignored.
4. **Project settings** → **Service accounts** → **Generate new private key**. Save it *outside the repo*, e.g. `~/secrets/love-button-sa.json` , mode 600. Open it and note `project_id` — you need it twice below.

THE TWO SILENT FAILURES IN THIS BLOCK

A wrong package name. The app builds fine, installs fine, and FCM simply never delivers. Nothing

anywhere says why.

A mismatched project id. `FIREBASE_PROJECT_ID` in `wrangler.toml` must match the service account's `project_id` character for character, because it is interpolated straight into the FCM URL. Get it wrong and every push is rejected by Google with a 404 that reads like a dead device token — which is exactly the confusion `isPermanentTokenFailure` was hardened against in section 03.

```
ls -l app/google-services.json && git check-ignore -v app/google-services.json
# both must succeed: the file exists AND git is ignoring it
```

Block C · Cloudflare

```
npm install -g wrangler
wrangler login

# C3 — create the two stores, copy BOTH ids out of the output
wrangler d1 create love-button
wrangler kv namespace create TOKEN_CACHE
```

Paste both ids into `server/wrangler.toml`, replacing the zero placeholders, and set `FIREBASE_PROJECT_ID` to the value from block B. While you are in the file, set `PERSON_1_NAME` and `PERSON_2_NAME` — they are what the app shows on each paired screen. **None of these three values are secrets**; they are safe in a public repo.

```
# C5 — the two enrolment codes, 48 hex characters each
openssl rand -hex 24          # ENROLL_CODE_1 — yours
openssl rand -hex 24          # ENROLL_CODE_2 — hers

# C6 — the three secrets. Write-only from here on.
cd server
wrangler secret put ENROLL_CODE_1
wrangler secret put ENROLL_CODE_2
cat ~/secrets/love-button-sa.json | tr -d '\n' | wrangler secret put FCM_SERVICE_ACCOUNT

# C7/C8 — schema onto the live database, then ship it
npm run migrate:remote
npm run deploy

curl https://love-button.<subdomain>.workers.dev/health # {"ok":true}
```

THREE THINGS THAT BITE HERE

The `tr -d '\n'` is not optional. The service-account JSON must arrive as a single line or the secret is unusable.

Put both enrolment codes in a password manager before you paste them. The server stores only a hash, and Cloudflare will never show you a secret again. Lose them and you cannot re-enrol a phone after a factory reset without deploying new ones.

Do not continue past a failing `/health`. Everything after this point assumes a reachable Worker, and diagnoses badly if it is not.

Prove the push path before writing a line of app code. Enrol a fake device with `curl`, then call `/v1/send`. If FCM accepts it, the hard half of the system is finished and everything after is layered onto something you have already watched work.

Block D · The phones

1. **Confirm both phones are Global or EEA ROM, not China ROM.** Settings → About phone → build number: Global ends `.MIXM`, EEA ends `.EUXM`, China ends `.CN`. A China ROM has no Play Services, so FCM cannot work there at all — no push, no workaround, project over. **Check this before anything else touches a phone.**
2. **Developer options.** On HyperOS the tap target was renamed: Settings → About phone → tap "OS version" seven times, not "MIUI version". The menu then lives under **Additional settings**, not System.
3. **USB debugging**, plus **Install via USB** — the one that fights back. It needs a Mi account signed in on the phone, and a *newly created* Mi account can be gated for up to **96 hours** with no workaround. Create the accounts first.
4. **USB mode must be File Transfer / MTP.** On Xiaomi, "Charging only" silently blocks adb — the device simply never appears.
5. Accept "Allow USB debugging?" on the phone and tick **Always allow from this computer**, or you re-accept on every reconnect.
6. Set `apiBaseUrl=https://love-button.<subdomain>.workers.dev` in `local.properties`. A missing value fails the build with that exact instruction, on purpose.

WHEN "INSTALL VIA USB" DEMANDS A SIM CARD

HyperOS validates that toggle against Xiaomi's servers and refuses without a SIM present. In order of preference: **(1) wireless debugging** — Developer options → Wireless debugging → pair with code, then `adb pair <ip>:<pairPort>` and `adb connect <ip>:<mainPort>` (note the two ports differ); it is a different code path, so the toggle does not apply and you get full adb. **(2) Any SIM** — the check is presence, not an active plan; an expired one works and the toggle usually stays on after you remove it. **(3) Sideload by hand** over MTP. *Not* the region-change-to-India trick: it works, but it moves an EEA device off EEA privacy defaults for no proportionate gain.

```
adb devices -l                # both serials, neither "unauthorized"
adb -s <serial> install app/build/outputs/apk/debug/app-debug.apk
# both phones can be plugged in at once; adb only needs -s to disambiguate
```

Block E · The four sounds, before any channel code runs

This block is *ordered* where it is for a reason covered in section 05: a channel's sound is frozen at creation, so creating `msg_1` with a placeholder burns that id permanently. Finalise all four first.

- **Where:** Pixabay sound effects (biggest library, no attribution), Mixkit, Kenney's Interface Sounds (CC0). Freesound only if you check each file individually — it mixes CC0 and CC-BY and the filter is not always right.
- **Under about one second.** Longer clips get truncated by the system anyway.
- **Distinct in timbre, not just length.** This is the criterion people get wrong. She will hear these from a pocket, muffled, without looking — and two different bells are the same bell through fabric. Aim for four different *kinds*: a soft chime, a wooden knock, a two-note rise, a low pluck.
- **Not like stock Android or WhatsApp.** A sound resembling her other notifications defeats the entire point.
- **Audition all four back to back, quietly, from another room.** That is the real listening condition, not headphones at a desk.

Then, in this order

1. **Minimal app** — enrol screen, one button, `PushService`. Nothing else. Stop when a tap on one phone buzzes the other.
2. **Leave it overnight.** Send one in the morning before building anything else. If that fails, nothing built on top of it is worth anything.
3. Then sounds, widgets, receipts, art — hardware-verified at each step.

VERSION NUMBERS

`versionCode` must be bumped on **every** build that goes to a phone — Android refuses an install over an equal or lower code, and the refusal names only `INSTALL_FAILED_VERSION_DOWNGRADE`, nothing that says "you forgot". `versionName` deliberately stays `1.0` forever: two people sharing an app do not need a version to talk about. The cost, stated up front: Settings shows 1.0 for every build, so `adb shell dumpsys package com.lovebutton.app | grep versionCode` is the only way to tell which one a phone has.

DEPENDENCY TRAPS, ALREADY PAID FOR

Two version pins in `libs.versions.toml` carry warnings because their failure modes name neither the library nor a version: Compose BOM `2026.08.00` demands AGP 9.1+ and compileSdk 37 and fails as 22 opaque AAR-metadata violations; Glance `1.3.0-alpha02` fails the same way. And `firebase-messaging-ktx` no longer exists — Google folded KTX into the main artifacts, so recent BOMs carry no version for it and the dependency resolves to a confusing "Could not find" error.

How the work actually moved

Seven milestones, each one a plan document, each plan built task by task against tests written first. The order below is not aspirational — it is what the repo's `docs/` and git history record.

STAGE	ARTIFACT	WHAT IT DECIDES
1 · Spec	<code>love-button-spec.md</code>	The binding authority, ~34KB. Sections for architecture, security, server, app, widgets, MIUI, build order, known traps. Crucially it has a "Deliberately not built" section and a "Deferred" section — it is mostly a record of what was cut and why.
2 · Design	<code>specs/<date>-...-design.md</code>	Per feature. Argues <i>from</i> the spec, and can propose a deviation — the client-minted <code>send_id</code> was exactly that, approved in the design doc before any code.
3 · Plan	<code>plans/<date>-...md</code>	Global constraints restated at the top, then numbered tasks. Each task names its files, its consumed and produced interfaces, and checkbox steps — with the failing tests written out in full in the plan itself.
4 · Preflight	<code>sdd/.../progress.md</code>	Before writing code: a conflict scan matrix of every task pair, "produces vs consumes", marked Clean or DEFECT. Defects become numbered rulings that override the plan text.
5 · Build	<code>task-N-brief / -report</code>	One task at a time, tests first. A brief in, a report out.
6 · Review	<code>review-<sha>..<sha>.diff< code=""></sha>.diff<></code>	Every task's diff saved and reviewed against the plan's constraints.
7 · Gate	real hardware	"Do not proceed until the current milestone is verified on real hardware." Overnight test before decoration.

The preflight scan is the unusual part

It catches integration defects *before* the code exists, when they cost a sentence instead of a refactor. Two real examples from the receipts plan:

- **Ruling 1:** two tasks each defined the 20-second window — one as `PENDING_WINDOW_MS`, one as a literal `20_000L`. Two sources of truth for one number means a later change silently desynchronises the tile from the map that governs it. Ruling: import the constant.
- **Ruling 2:** a task's plan text contained `Result.success().takeIf { ok } ?: Result.success()` — a dead expression dressed as a decision. Caught by reading, never run.

Build order, as actually followed

#	MILESTONE	WHAT YOU CAN DO AT THE END OF IT
0	Accounts, keys, project skeleton	—
1	Worker: <code>/health</code> , <code>/v1/enroll</code> , <code>/v1/send</code>	Trigger a push with a single <code>curl</code>
2	Minimal app: enrol screen, one button, receiving service	Tap your phone, hear hers buzz
3	MIUI checklist screen + overnight smoke test	Trust it
4	Four messages, four channels, four sounds	Hear the difference between them
5	One widget: send, haptic, idle/sending/sent/failed	Send from the home screen
6	Receipts: endpoint, reverse push, 20s timeout, seen-on-unlock	See it land
7	The other three widgets	—
8	Hardening: abuse ceiling, retention cron, <code>UNREGISTERED</code> cleanup	Publish it

Milestone 2 is the whole hard part. Everything after it is decoration layered onto something you have already watched work.

One habit worth stealing

Read any file in this repo and the comments do not explain *what* the code does — they explain what was tried, what broke, and why the current shape is the one that survived. `fcm.ts` spends fifteen lines on why `INVALID_ARGUMENT` is not trusted. `TimeoutWorker` explains why it has no network constraint. That is the difference between code you can maintain in six months and code you have to re-derive. The commit messages carry the same voice: *"only UNREGISTERED means the token is dead", "a send reclaims the bubble from an older message that jumped it"*.

11 — LOOK THESE UP

General ideas this project happens to use

Roughly in the order you meet them above. None of these are quirks of this project; all of them transfer.

SERVER

- **Bearer tokens** — why a random string in a header is enough to prove identity, and why the server stores only its hash
- **Constant-time comparison** — how naive string comparison leaks secrets through timing

- **JWT and RS256** — what a signed token is; why the Worker signs one to get another
- **OAuth 2.0 service accounts** — machine-to-machine auth with no human logging in
- **Edge compute and cold starts** — why a Worker has a 10ms CPU budget at all
- **Idempotency** — why `/v1/receipts` must tolerate being called twice

PLATFORMS

- **Push architectures** — why every mobile OS funnels wake-ups through one vendor socket, and what that means for anything you build on top
- **Bindings vs connection strings** — infrastructure injected as objects, and the class of secret it removes entirely
- **Eventual consistency** — when a slightly stale read is fine (a rate-limit counter) and when it is not (an auth lookup)
- **Write-only secrets** — why a store you cannot read back is a feature, and what it forces on your backup strategy

ANDROID

- **Doze and App Standby** — how Android decides to stop your app running
- **High-priority FCM** — the exemption that lets a message punch through Doze
- **Notification channels** — the Android 8 model that moved control from app to user
- **PendingIntent** — handing another process permission to act as you
- **WorkManager** — durable background work that survives process death
- **RemoteViews and IPC** — why widgets are so much more constrained than normal UI

GENERAL

- **Threat modelling** — the question is always "who is the attacker?"
- **Defence in depth** — the rate limit exists even though the codes are already unguessable
- **Total ordering** — why an arbitrary tie-break beats no tie-break, as long as both sides compute it identically
- **YAGNI** — the spec is mostly a record of what was cut

KEEPING THIS ON YOUR PHONE

As a PDF: open this page on your phone, then share sheet → **Print** → **Save as PDF**. The page has print styling, so it lays out as a clean document rather than a screenshot of a website.

As a file: the standalone `.html` was sent to you in the conversation — save it to Files or Drive and it opens in any browser, offline. Only the two web fonts need a connection; without one it falls back gracefully and stays fully readable.

